Scrabble en Java

Alexandre Lou-Poueyou
Aymeric Cassard
Ludovic Andrieu
Jean Mayol
Mathéo Petry
Valentin Cazet

3 avril 2025

Table des matières

1 Le contexte et l'existant du projet **3**
1.1 Choix du jeu et du langage 3
1.1.1 Pourquoi le scrabble ? 3
1.1.2 Java 3
1.2 Existant du projet 4

2 La liste des besoins réalisés **5**

3 Une explication détaillée du sujet **9**
3.1 Règles du jeu 10
3.1.1 Installation de la partie 10
3.1.2 déroulement de la partie 10
3.1.3 fin de la partie 11
3.1.4 comptage des points 11
3.1.5 Variantes du jeu 11
3.2 Structures de données 12
3.3 Algorithmes spécifiques 13

4 Architecture du projet **13**
4.1 Modules du projet 13
4.2 Dépendances du projet 14

5 Les tests réalisés **14**

6 Une analyse critique du projet **15**

1 Le contexte et l'existant du projet

TARGET :
LE CONTEXTE ET L'EXISTANT DU PROJET ;
(SUJET CHOISI, LANGAGE CHOISI, BIBLIOGRAPHIE ASSOCIÉE AVEC AU MOINS 10 RÉFÉRENCES)
STATE : relecture en cours par val

1.1 Choix du jeu et du langage

1.1.1 Pourquoi le scrabble ?

Notre choix pour le projet s'est porté sur le Scrabble. Cela nous a semblé être le meilleur choix pour notre groupe car ce jeu est très répandu et ses règles assez basiques permettent une compréhension facile du sujet. De plus, celles-ci ne possèdent pas de cas particuliers complexes à d'autres jeux comme les échecs ou le Tafl [4], ce qui nous permet de nous concentrer sur des aspects plus techniques de la gestion et création du projet (Génie Logiciel, Interface Graphique, bon déroulement du jeu).

1.1.2 Java

Le langage de programmation ayant retenu notre attention est Java grâce à sa richesse, son accessibilité et sa lisibilité, de par sa documentation [1] très complète et ses nombreuses librairies robustes qui peuvent nous simplifier diverses tâches. De plus, nous avons comparé Java avec les autres langages disponibles (Python, C) et nous avons repéré des points négatifs dans les autres langages :

Python :

- Langage plus lent [2]
- Importations des fichiers plus sujets à problème
  TODO : reformuler, on ne comprend pas la source du probleme
- Typage spécifique de Python pouvant être plus compliqué qu'autre chose vis à vis de la taille du projet
  TODO : reformuler, il faut expliquer !

TODO : il faut plus expliquer !
c'est pas une liste a puce, c'est un rapport FORMEL
C :

- Aucune structure de données implémentées de base

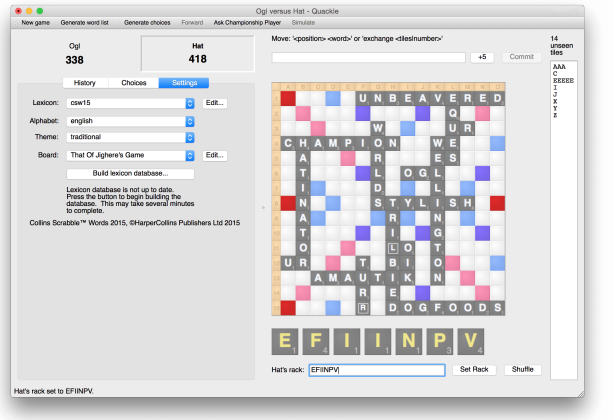
— Gestion de la mémoire

— Pas de possibilité de faire de la programmation orienté objet

1.2 Existant du projet

TODO : reformuler platement + add [?]

Le scrabble étant un jeu très connu et répandu, il n'est pas étonnant de trouver des existants de ce jeu, principalement pour ce qui est du test de performances d'algorithmes. On peut retrouver dans cette situation le Quackle [3] développé par le MIT et étant une implémentation du scrabble avec une IA utilisant l'algorithme Gaddag pour lui permettre de jouer le meilleur coup possible.



Mais on peut aussi retrouver des implémentations du Scrabble se concentrant sur l'aspect ludique et distrayant via l'ajout de différents modes de jeu, d'un aspect compétitif ou d'éléments de customisation. Par exemple, les jeux de Scrabble développés pour les mobiles.

TODO : passer le texte du dessous en tant que légende de l'image

Scrabble Go sur mobile :



2 La liste des besoins réalisés

TARGET :
LA LISTE DES BESOINS RÉALISÉS ;
(BESOINS EFFECTIVEMENT RÉALISÉS PENDANT LE PROJET)

TODO : remplacer par une image ou un tableau avec checkmark ?

— **F04 : Systeme cible**
Le programme devra fonctionner sur des systèmes d'exploitations GNU/Linux

— **F07 : Bugs**
Aucun bug menant à un crash ne doit être présent dans le programme. Les spécifications doivent être respectées et les erreurs doivent générer des messages d'erreur.

— **F10 : Frameworks de tests**
Le framework de test sera basé sur : • Langage C : CTest + googletest + gcovr. • Langage Java : JUnit + Jacoco. • Langage Python : pytest + coverage.

— **F11 : Gestion des options**
Les options en ligne de commande seront gérées par : • Langage C : le module getopt long. • Langage Java : la bibliothèque commons-cli. • Langage

Python : le module argparse.

— **F12 : Bibliothèque graphique**
La bibliothèque graphique utilisé sera basé sur : • Langage C : LibGtk 4.x. • Langage Java : JavaFx. • Langage Python : PyGObject

— **F13 : Internationalisation**
Le framework d'internationalisation sera basé sur : • Langage C : la bibliothèque gettext. • Langage Java : ResourcesBundle et Locale. • Langage Python : le module Python gettext. La langue par défaut, sans locale précisée, sera l'anglais et, au moins, le français sera ajouté.

— **F14 : Nom de l'exécutable principa**
L'exécutable principal doit s'appeler 'scrabble' et permettre de lancer tout le programme.

— **F15 : Usage général**
Le programme sera lancé comme ceci :
#> scrabble [OPTIONS] [FILENAME] Si une option est incorrecte, le programme devra afficher un message d'erreur et l'aide du programme. Si un nom de fichier est donné, le programme devra charger une partie depuis ce fichier. Sinon, le programme devra démarrer une nouvelle partie. Les messages à l'utilisateur seront écrits sur la sortie standard (stdout) et les messages d'erreurs devront être écrits sur la sortie d'erreur standard (stderr). De plus, le code de retour du programme devra être EXIT SUCCESS en cas de succès et EXIT FAILURE en cas d'erreur

— **F16 : Aide en ligne de commande**
L'option '-h', '—help' affiche l'aide puis quitte.

— **F17 : Version**
L'option '-V', '—version' affiche la version puis quitte

— **F18 : Mode verbose**
L'option '-v', '—verbose' permettra d'afficher des informations supplémentaires pendant l'exécution.

— **F19 : Mode debug**
L'option '-d', '—debug' permettra d'afficher des informations de débogage

pendant l'exécution.

— **F22 : Nombre de joueurs**
L'option '-p N', '—players N' permettra de spécifier le nombre de joueurs pour une partie (de 2 à 4). Par défaut, le jeu se jouera à deux joueurs.

— **F23 : Langue du jeu**
L'option '-l LANG', '—language LANG' permettra de spécifier la langue du jeu. Par défaut, le jeu sera en anglais (en). Les langues supportées seront au moins : anglais (en), français (fr). Il devra être possible de lister les langues supportées avec l'option '—list-languages'.

— **F24 : Mode Super-scrabble**
L'option '-s', '—super' permettra de jouer en mode Super-scrabble. Dans ce mode, le plateau de jeu sera plus grand (21 par 21) et le nombre de lettres est augmenté.

— **F25 : Dictionnaire personnalisé**
L'option '-D FILENAME', '—dictionary FILENAME' permettra de spécifier un fichier de dictionnaire personnalisé. Par défaut, le programme utilisera le dictionnaire fourni avec le programme.

— **F30 : Affichage du plateau**
À chaque coup pour un humain, le programme devra afficher le plateau ainsi que le temps pris par chaque joueur si la partie est en blitz. En mode ASCII les cases vides seront représentées par un 'underscore' et les lettres par leur lettre majuscule. Les cases spéciales seront mises en exergue par des couleurs différentes en utilisant les codes ANSI pour changer la couleur de background. Les cases 'lettre compte double' seront cyans, les cases 'lettre compte triple' seront bleues, les cases 'mot compte double' seront rouge et les cases 'mot compte triple' seront violettes. Dans le cas du Super-scrabble, les cases 'lettre compte double' seront cyans léger (light cyan), 'lettre compte triple' seront cyans, les cases 'lettre compte quadruple' seront violettes, les cases 'mot compte double' seront rouges léger (light red), les cases 'mot compte triple' seront rouges et les cases 'mot compte quadruple' seront violettes.

— **F31 : Nommer les joueurs**
Les utilisateurs pourront remplacer leur numéro de joueur par un nom à tout

moment de la partie. C'est ce nom qui sera affiché à l'écran pour indiquer le joueur courant et les scores.

— **F32 : Placement d'un mot sur le plateau**
Le programme devra permettre de placer un mot composé à partir de ses lettres sur le plateau. Le joueur devra entrer les coordonnées de la case de départ, la direction du mot (horizontale ou verticale) et le mot à placer. Par exemple, pour placer le mot 'COUNT' en partant de la case g5 en direction verticale, le joueur entrera : 'g5v COUNT'. Le programme devra vérifier que le mot peut effectivement être composé à partir de la réserve de lettres du joueur, que les cases sont bien vides là où il faut et qu'il est bien présent dans le dictionnaire de la partie. Si la proposition du joueur est incorrecte, un message d'erreur qui indiquera la cause du problème devra être affiché et le programme devra demander une autre action.

— **F33 : Passer son tour**
Le programme devra permettre à un joueur de passer son tour s'il ne peut pas ou ne veut pas jouer de mot.

— **F34 : Échange de lettres**
Le programme devra permettre à un joueur d'échanger un certain nombre de lettres de sa réserve contre un nombre équivalent de lettres prises dans le sac. Le joueur devra entrer les lettres à échanger. Par exemple, pour échanger les lettres 'A', 'B' et 'C', le joueur entrera : 'ABC'. Le programme devra vérifier que les lettres sont bien présentes dans la réserve du joueur et qu'il reste suffisamment de lettres dans le sac. Si la proposition du joueur est incorrecte, un message d'erreur qui indiquera la cause du problème devra être affiché et le programme devra demander une autre action.

— **F35 : Gestion du sac de lettres**
Le programme devra gérer un sac de lettres qui contiendra toutes les lettres du jeu. À chaque fois qu'un joueur pioche des lettres, elles devront être retirées du sac. Si le sac est vide, le programme devra afficher un message et la partie sera terminée

— **F36 : Abandon**
Le programme devra permettre à un joueur d'abandonner la partie en cours.

— **F37 : Comptage et affichage des scores**
Le programme devra afficher les scores des joueurs à chaque coup. Les

scores attribués à chaque lettre dépendent de la langue utilisée, le programme devra être capable de gérer plusieurs langues et donc plusieurs valeurs de lettres. Par défaut, le programme utilisera les valeurs des lettres qui correspondent à la locale de l'utilisateur qui lance le programme ou bien l'anglais si la langue n'est pas supportée

— **F41 : Recommencer une partie**
À tout moment, il doit être possible de recommencer une partie depuis le début en abandonnant la partie en cours.

— **F45 : Fichier de configuration**
Le programme devra être capable de lire un fichier de configuration (.scrabblerc) qui contiendra les options par défaut du programme. Ce fichier sera lu au démarrage du programme et les options qu'il contient seront utilisées par défaut. Les options passées en ligne de commande devront écraser les options du fichier de configuration. On utilisera le format INI pour ce fichier

— **F46 : Dictionnaire**
Le programme devra comporter un module lexicon qui permettra de charger un dictionnaire de mots. Ce dictionnaire sera utilisé pour vérifier si un mot est valide. Le format d'entrée du dictionnaire sera un fichier texte contenant un mot par ligne. Le programme devra charger ce dictionnaire au démarrage en fonction de la langue choisie par l'utilisateur. Par défaut, le programme chargera le dictionnaire qui correspond à la locale de l'utilisateur s'il possède un dictionnaire pour cette langue, sinon il chargera le dictionnaire anglais.

— **F51 : Menu 'About'**
L'interface graphique devra comporter un menu d'options qui permettra de consulter des informations sur le programme (version, auteurs, ...).

3 Une explication détaillée du sujet

TARGET :
Une explication détaillée du projet final ;
(règles du jeu, structures de données employées, algorithmes spécifiques, etc.)

3.1 Règles du jeu

Le Scrabble est un jeu de société se jouant de 2 à 4 joueurs et qui consiste à former des mots sur une grille carrée de 15×15 cases. Le but du jeu est de marquer le plus de points en posant des mots. Le jeu du Scrabble contient un plateau de jeu, un ensemble de jetons de lettres variant selon le pays, un sac de lettres et des chevalets de jeu permettant de poser les jetons tout en cachant ses lettres aux adversaires.

3.1.1 Installation de la partie

On met tous les jetons de scrabble dans le sac de lettres. On pose au centre de la table le plateau de jeu et devant chaque joueur des chevalets de jeu. Chaque joueur pioche sans regarder une lettre dans le sac de lettres. Celui qui a la lettre la plus haute dans l'alphabet (A > B, B > C, etc.) commence. Si un joueur pioche le joker, il doit repiocher. Ils remettent toutes les lettres quand le 1er joueur est déterminé.
Le 1er joueur pioche ensuite 7 lettres dans le sac de lettres et les garde sans les montrer aux adversaires. Les autres joueurs font de même dans le sens horaire.
La partie peut désormais commencer.

NOTE pour val : reprendre ici

3.1.2 déroulement de la partie

Au début de la partie, le 1er joueur pose sur le plateau un mot qu'il peut former à partir des lettres de son jeu. Une lettre du mot doit être posée sur le centre du plateau (la case de coordonnée h8 pour un plateau normal). Il pioche ensuite dans le sac de lettres un nombre de lettres égales au nombre de lettres posé sur le plateau. Il pioche donc jusqu'à avoir 7 lettres sur son chevalet.
Le 2ème joueur fait de même mais doit par contre rattacher son mot au mot du 1er joueur (attention il ne peut superposer ces lettres sur le mot du 1er joueur). Il faut bien évidement que le mot rattaché au mot du 1er joueur soit aussi légal. Il fait pareil que le 1er joueur concernant la pioche. La partie se déroule ainsi de suite jusqu'à la fin.
Les seules actions que peut entreprendre le joueur lors de son tour sont :

— Poser un mot.

— Échanger des lettres de la pioche.

— Passer son tour.

3.1.3 fin de la partie

Les conditions de fin de partie sont les suivantes :

— **Reliquat épuisé** : Un joueur a épuisé toutes ses lettres et la pioche est vide. C'est la fin basique d'une partie normal. Les joueurs qui n'ont pas terminés retirent de leurs scores la valeur des lettres qu'ils n'ont pas posés et le joueur qui a finit ajoute a son score l'ensemble de ces valeurs ainsi retirés.

— **Blocage** : Les joueurs ne trouvent plus de mots a jouer et il reste moins de 7 lettres dans la pioche. Les valeurs des lettres de leur mains sont déduites de leur score final.

Le joueur avec le score le plus élevé gagne.

3.1.4 comptage des points

La grille contient des cases bonus :

— Lettre double : double la valeur de la lettre posée.

— Lettre triple : triple la valeur de la lettre posée.

— Mot double : double la valeur du mot posé.

— Mot triple : triple la valeur du mot posé.

Lorsqu'un joueur pose des lettres, le score est calculé automatiquement. Le score du coup est la somme des valeurs des lettres de chaque mot formé par le coup, majorée des points obtenus en plaçant des lettres sur des cases multiplicatrices. On applique les bonus de lettres avant d'appliquer celui des mots. Si plusieurs mots sont formés occupant la même case multiplicatrice d'un seul coup, le joueur obtient le bonus correspondant à chaque mot formé. Une fois qu'une case multiplicatrice a été utilisée, elle ne rapporte plus de points si un autre mot est formé ultérieurement avec la tuile qui l'occupe.

Si un joueur pose toutes ses lettres en un tour, il réalise un "scrabble" et score donc un bonus de 50 points.

3.1.5 Variantes du jeu

Il existe plusieurs variantes au scrabble, dont :

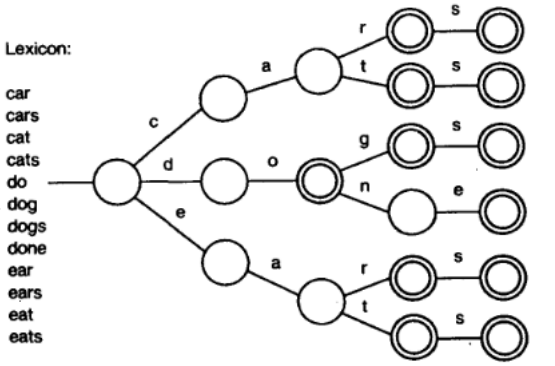
— **Super-scrabble** : La grille fait 21x21.

— **Blitz** : Chaque joueur dispose d'un temps imparti, qui s'écoule lors de son tour. Si un joueur n'a plus de temps, il perd.

3.2 Structures de données

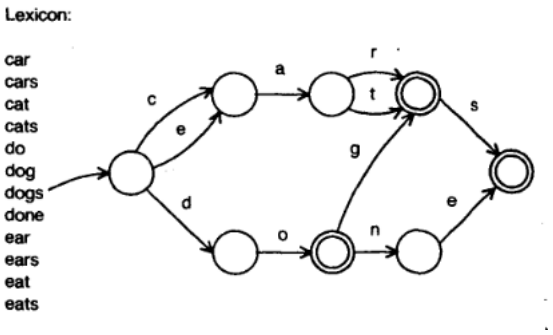
Structures de données simples
L'utilisation de Java dans le projet permet d'éviter d'implémenter par nous-mêmes les structures de données de base du projet. Par exemple, la pioche est représentée par une HashTable qui associe chaque lettre à un compteur, représentant le nombre de lettres restantes dans la pioche.

Représentation du dictionnaire
Une manière commune de représentation de lexique est un arbre pour lequel chaque arête est une lettre du mot, et certains nœuds sont marqués comme finaux, indiquant que le chemin jusqu'à ce nœud constitue un mot. Cette structure est communément appelée un trie.



Le trie n'est pas utilisé dans notre projet mais aide à visualiser le fonctionnement des structures de données suivantes.

Le **DAWG**(Directed Acyclic Word-Graph) [3] nous permet de représenter le dictionnaire de validité des mots en mémoire. Il s'agit d'un automate fini pour lequel le langage est l'ensemble des mots du dictionnaire, et les transitions sont les lettres que l'on ajoute pour former le mot. Enfin, les états finaux sont les différents mots du lexique.



Cette structure de données est équivalente au trie en termes de fonctionnalités, mais prend moins de place en mémoire.

3.3 Algorithmes spécifiques

4 Architecture du projet

TARGET :
L'ARCHITECTURE FINALE DU PROJET ;
(ARCHITECTURE FINALE, EN FAIRE UNE CRITIQUE, DONNER DES VOIES D'AMÉLIORATIONS)

4.1 Modules du projet

Nous avons choisi une architecture de style MVC pour organiser les modules du projet. Ce motif d'architecture a été choisi car il pose des lignes directrices simples et comprises par tous les membres vis-à-vis de la question du placement des différents modules. De plus, il se prête bien à la mise en place de la partie graphique. Conformément à ce modèle, nos modules sont donc découpés en plusieurs parties.

Modèle

Le modèle contient les différentes structures de données, et autres éléments logiques relatifs aux contraintes du jeu. Les éléments de cette couche correspondent ce qui est nécessaire à la Game (joueurs, lettres, multiplicateur, ...)

Vue

La vue est constituée de tous les éléments visibles par l'utilisateur. Ici, elle est découpée en deux grands axes. La partie CLI, et la partie GUI.

Controleur

Cette couche de l'architecture traite les interactions avec l'utilisateur. C'est donc cette partie qui s'occupe d'actualiser le modèle selon les différentes actions de

l'utilisateur.
Erreurs
On renvoie un message spécifique à l'erreur et cela pour chaque erreurs du jeu (par exemple dépassement de la grille quand on pose une lettre). On rassemble ainsi tout les erreurs rencontrés dans cette couche.
Dictionnaire
C'est ici qu'est réalisé le dwag en plus de vérifier les lettres valides au scrabble.
Configuration
Important afin de pouvoir lancer correctement le jeu selon certaines fonctionnalités tels que les modes de jeu, du nombre de joueur, de la langue, ...

4.2 Dépendances du projet

Le projet est codé en Java version 17.0.13, et est compilé à l'aide du système de build Maven. Afin de respecter les spécifications du projet, un certain nombre de dépendances sont ajoutées :

- **JUnit** : Framework de test, utilisé pour les différents scénarios de test du code.
- **Jacoco** : Librairie de couverture de code, utilisée en conjonction avec JUnit pour certifier le pourcentage de code source testé.
- **Commons-cli** : API d'analyse d'options de commandes. Utilisé pour interpréter les options passées en ligne de commande.
- **JavaFx** : Framework d'interface utilisateur, utilisé pour la partie interface graphique.
- **Spotless** : Outil d'analyse de code permettant de vérifier la conformité du formatage avec le style de codage de Google.

Grâce à une gestion et vision précise du projet réalisé par un de nos membres, nous ne nous sommes pas perdus sur l'architecture du projet. En revanche il est arrivé que nous ayons dû faire des choix et des modifications soudaines de cette architecture au fur et à mesure de l'avancement du projet. De plus certaines parties du projet aurait sûrement pu être plus fracturer afin qu'il soit plus lisible et compréhensible. Prendre plus de temps, faire des points pour restructurer si nécessaires semble être bénéfique pour un projet de cette envergure.

5 Les tests réalisés

TARGET : NEW SECTION
LES TESTS RÉALISÉS DANS LE PROJET.

(LES TESTS RÉALISÉS POUR VALIDER CHACUNE DES BESOINS VISÉS)

6 Une analyse critique du projet

TARGET :
UNE ANALYSE CRITIQUE DU PROJET ;
(PERFORMANCES, CAS LIMITES, LIMITATIONS, BUGS, AMÉLIORATIONS POSSIBLES, ETC.)

Le projet bien qu'il marche et qu'on peut y jouer normalement, manque de plusieurs fonctionnalités tels que l'implémentation d'une l'IA. On peut jouer directement depuis la console et on a une interface graphique fonctionnelle. On est malheureusement assez limités sur certaines fonctionnalités qu'on aurait voulu améliorer mais qui sont compliqués a mettre en place et de plus ne sont pas forcément prioritaire. Par exemple le mode multijoueur sur plusieurs PC, un système de classement en fonction du score le plus élevé. On a pris trop a la légère certaines parties du projet et avons pris ainsi du retard qui s'est répercutés à la fin du projet. Nous avons donc choisi de prioriser un code propre et efficace, sans bug que nous précipiter pour réaliser le plus de fonctionnalités. On a manqués d'anticipations et nous devrons être plus vigilent a cela. Malgré tout, l'implémentation d'une IA et de multiples fonctionnalités afin de perfectionner le projet sont les axes principaux d'améliorations futurs d'un projet tel quel.

La correction linguistique a été effectuée par ChatGPT d'OpenAI [5].

Références

[1] O. and/or its affiliates. *Java SE 17 Documentation*, 2024.

[2] P. L. Benchmarks. Java vs python benchmarks, 2025.

[3] A. W. A. . G. J. Jacobson. The world's fastest scrabble program. *Programming Techniques and Data Structures*, 31 :572–578, May 1988.

[4] P. nordiques. Tafl, Probablement entre le 4ème et le 12ème siècle. Ancien jeu de stratégie, originaire des cultures nordiques.

[5] OpenAI. Correction linguistique effectuée par chatgpt. Disponible en ligne : `https://www.openai.com/chatgpt`, 2025. Correction linguistique réalisée à l'aide de l'outil ChatGPT d'OpenAI.